Estimation of the Log Gaussian Cox Process Model with Point and Aggregated Case Data: The rts2 Package for R

by Samuel I. Watson

Abstract Log Gaussian Cox Process (LGCP) models are a key method for disease and other types of surveillance, and provide a means of predicting risk across an area of interest based on spatially-referenced and time-stamped case data. However, these models can be difficult to specify and computationally demanding to estimate. For many surveillance scenarios we require results in near real-time using routinely available data to guide and direct policy responses, or due to limited availability of computational resources. There are limited software implementations available for this real-time context with reliable predictions and quantification of uncertainty. The **rts2** package provides a range of modern Gaussian process approximations and model fitting methods to fit the LGCP, including estimation of covariance parameters, using both Bayesian and stochastic Maximum Likelihood methods. The package provides a suite of data manipulation tools. We also provide a new implementation to estimate the LGCP when case data are aggregated to an irregular grid such as census tract areas.

1 Introduction

The statistical analysis of geolocated and possibly time-stamped data representing cases of a phenomenon of interest can help identify areas with a high probability of high or growing risk. In this article, we focus primarily on disease surveillance, but other applications include social policy, crime, ecological, geological, or other point process phenomena. In the context of disease surveillance, prevalence mapping surveys are a rigorous method for measuring disease risk and conduct a 0/1 test on a sample from a population in a given area to predict the spatial distribution of risk (Diggle and Giorgi, 2016). Recent examples include the mapping of seroprevalence during the SARS-CoV-2 pandemic (e.g. Riley et al. (2020)). However, such studies require significant time and resources. In many contexts there is a need for more rapid analyses of case data to be able to quickly identify and respond to emerging disease outbreaks. Again, SARS-CoV-2 provides a good example (Watson et al., 2021), but in many low- and middle-income country settings identifying emerging clusters of diseases like cholera, malaria, or influenza can facilitate targeted intervention to prevent the development of an epidemic (e.g. Ratnayake et al. (2020b,a)). Beyond emergency applications, reliable quantification of disease risk across an area of interest using routine data provides a useful tool for public health surveillance and research. Obtaining predictions in a reasonable amount of time without using excessive computational resources can facilitate their adoption.

Many healthcare and public health systems collect finely spatially-resolved and time-stamped data that can be used to monitor disease epidemiology. These data may include daily hospital admissions, positive tests, or calls to public health telephone services (Diggle et al., 2005). Case data are often also provided as counts at a spatially aggregated level, such as census tracts. Geospatial statistical models provide a principled basis for generating predictions of disease epidemiology and its spatial and temporal distribution from these data.

The **rts2** package provides data manipulation and model fitting methods for when these data can be assumed to be a realisation from a spatio-temporal point process over the area of interest. A point, such as the residential address of a hospital admission on a particular date, is assumed to be more likely to be observed in areas of higher risk than lower risk. Analysis of these data can provide probabilistic predictions about the underlying disease risk and be used to quantify our uncertainty about the presence of "hotspots".

A growing use case for spatio-temporal point process models is "real-time" disease surveillance that uses a stream of real-time data, such as hospital admissions, to provide rapid feedback to public health authorities. However, geospatial models can be computationally intensive and slow to run, and they can scale poorly with the size of the data. 'Fast' methods like kernel-based mapping require often difficult selection of bandwidth parameters and may have less reliable quantification of uncertainty. **rts2** provides a range of Gaussian process approximations and approximate inference, including both Bayesian and Frequentist approaches, to support faster and more scalable analyses for real-time scenarios. We first describe the statistical models and model fitting and other relevant packages for R, and then provide a discussion of the use of the package and generation and interpretation of outputs.

2 The log gaussian cox process

Our statistical model is a Log Gaussian Cox process (LGCP) (Diggle et al., 2013), whose realization is observed on the Cartesian area of interest $A \subset \mathbb{R}^2$ and time period of interest $T \subset \mathbb{N}$ (we assume discrete time periods). The resulting data are realisations of an inhomogeneous Poisson process with stochastic intensity function $\{\lambda(s,t) : s \in A, t \in T\}$. The number of cases occurring in any locally finite random set $S \subseteq A$ is Poisson distributed conditional on $\lambda(s,t)$:

$$Y(S,t) \sim \text{Poisson}\left(\int_S \lambda(s,t)ds\right) \tag{1}$$

The intensity is decomposed as the log-linear model:

$$\lambda(s,t) = r(s,t)\exp(X(s,t)\gamma + Z(s,t)) \tag{2}$$

where $r(s,t)$ is a spatially or spatio-temporally varying Poisson offset, typically the population density. $X(s,t)$ is a length Q row vector of spatially, temporally, or spatio-temporally varying covariates including an intercept and $\{Z(s,t) : s \in A, t \in T\}$ is a latent field.

We use an auto-regressive specification for the latent field:

$$\begin{aligned} Z(s,1) =& (1+\rho^2)^{-1}\mathcal{Z}_1(s) \\ Z(s,t) =& \rho Z(s,t-1) + \mathcal{Z}_t(s) \text{ for } t > 1 \end{aligned} \tag{3}$$

where ρ is the auto-regressive parameter. This specification, sometimes referred to as a "spatial innovation" model, facilitates computationally efficient model fitting as we describe in the subsequent section. We include both Bayesian and maximum likelihood model fitting approaches in the package and, depending on the paradigm, the spatial innovation term $\{\mathcal{Z}_t(s) : s \in A\}$ is given a Gaussian process prior distribution, or is itself a Gaussian process. In both cases, the realisations of $\mathcal{Z}_t(s)$, which we notate as $\mathbf{z}$ are multivariate Gaussian distributed with mean zero and covariance Σ. We use a minimally parameterised covariance function for the Gaussian process to define the elements of Σ:

$$\text{cov}(\mathcal{Z}_t(s), \mathcal{Z}_t(s')) = \sigma^2 h(||s-s'||;\phi) \tag{4}$$

where $||.||$ is the L2-norm. The package includes, for example, the squared exponential covariance function

$$h(||s-s'||;\phi) = \exp\left(\frac{-(||s-s'||)^2}{\phi^2}\right) \tag{5}$$

and the exponential covariance function

$$h(||s-s'||;\phi) = \exp\left(\frac{-||s-s'||}{\phi}\right) \tag{6}$$

where ϕ is the spatial range (or length scale) parameter.

The LGCP is complex to estimate as the marginal likelihood does not have a closed form solution. We include methods and approximations where there do not currently exist implementations in other software packages, which are discussed in Section 7.

Model fitting in this package uses the standard practice of dividing the area of interest into a regular grid (Diggle et al., 2013; Taylor et al., 2013, 2015) with grid cells $S_i : i = 1, ..., n$. The size of the grid cells should permit the assumption that the latent field Z is approximately constant within each cell. The counts in each of the n grid cells comprise our data $\mathcal{D} = \{Y(S_i, t) : i = 1, .., n; t = 1, ..., T\}$ such that our model is:

$$\begin{aligned} Y(S_i, t) &\sim \text{Poisson}(\lambda(S_i, t)) \\ \lambda(S_i, t) &= r(S_i, t)\exp(X(S_i, t)\gamma + Z(S_i, t)) \end{aligned} \tag{7}$$

where Z is as specified in Equation (3). For the Bayesian approaches we also require specification of priors and hyperpriors for the remaining model parameters as we discuss below.

For a given data set $\mathcal{D}$, we write the nT-length vector of realisations of the Gaussian process model as $\mathbf{z}$. We can then write the likelihood of the model parameters as

$$L(\gamma, \theta|\mathcal{D}) = \int \prod_{i=1}^{n}\prod_{t=1}^{T} f_{Y|z}(Y(S_i, t)|\gamma, \mathbf{z}) dF_{z|\theta}(\mathbf{z}|\theta) \tag{8}$$

where $f_{Y|z}$ is the Poisson probability density function with intensity given in (7) and $f_{z|\theta}$ is the multivariate Gaussian density with parameters $\theta = [\sigma^2, \phi, \rho]$. The components of the log-likelihood are therefore:

$$\begin{aligned} \log(f_{Y|z}(Y(S_i, t)|\gamma, \mathbf{z})) &= Y(S_i, t)\eta(S_i, t) - \lambda(S_i, t) - \log(Y(S_i, t)!) \\ \log(f_{Z|\theta}(\mathbf{z}|\theta)) &= -\frac{nT}{2}\log(2\pi) - \frac{1}{2}\log(|\Sigma|) - \frac{1}{2}\mathbf{z}^T\Sigma^{-1}\mathbf{z} \end{aligned} \tag{9}$$

where $\eta(S_i, t) = \log(r(S_i, t)) + X(S_i, t)\gamma + Z(S_i, t)$. The autoregressive specification of the latent field means we can partially reduce the complexity of the log likelihood. We can write:

$$\Sigma = P \otimes \Sigma_0$$

where Σ_0 is the covariance matrix for a single time period and

$$P = \begin{bmatrix} 1 & \rho & \rho^2 & \dots & \rho^T \\ \rho & 1 & \rho & \dots & \rho^{T-1} \\ \vdots & & \ddots & \vdots & \vdots \\ \rho^T & \rho^{T-1} & \rho^{T-2} & \dots & 1 \end{bmatrix}$$

Letting $\mathbf{z}_{[t]}$ refer to the elements of $\mathbf{z}$ for time period t and $\mathbf{v}$ be an $n \times T$ matrix formed by stacking the elements of $\mathbf{z}$ into columns with column t equal to $\mathbf{z}_{[t]}$, we can then write:

$$\begin{aligned} \mathbf{z}^T\Sigma^{-1}\mathbf{z} &= \mathbf{z}^T(P \otimes \Sigma_0)^{-1}\mathbf{z} \\ &= \mathbf{z}^T\text{Vec}(\Sigma_0^{-1}\mathbf{v}P^{-1}) \\ &= \sum_t \mathbf{z}_{[t]}\Sigma_0^{-1}\tilde{\mathbf{v}}_{[t]} \end{aligned}$$

where $\tilde{\mathbf{v}} = \mathbf{v}P^{-1}$. Similarly $\log(|\Sigma|) = n\log(|P|) + T\log(|\Sigma_0|)$ Computational time with this approach is therefore linear in the number of time periods. However, evaluation of Σ_0^{-1} is still computationally demanding and scales as $O(n^3)$, which may still be prohibitive for many real-time surveillance applications.

2.1 Model for spatially-aggregated case data

The models discussed so far have assumed that the precise location of each case is known. However, in many cases we will have instead case counts aggregated to census or administrative areas for reasons such as preserving anonymity. These areas are almost always irregularly shaped and have a large variability in area. As a result, we cannot necessarily assume that the underlying latent process is approximately constant within each area, as we do with the regular lattice, which can cause bias. We can instead extend the LGCP model described above to account for the irregular areas onto which case counts are aggregated.

Similar to Li et al. (2012a), we notate the count data for this 'region' model as $Y(R_j, t)$ where R_j is the jth region (i.e. irregular area) of r total regions. The intersection areas between the regions and the computational grid are $Q_{ij} = R_j \cap S_i$ where there are q non-empty intersection areas. Letting $\boldsymbol{\lambda}_s$ be the $n \times T$ vector of grid-level intensities in (7), and $\boldsymbol{y}_r$ be the $r \times T$ vector of region level outcome variables. We consider models of the form:

$$\boldsymbol{y}_r \sim \text{Poisson}(W\boldsymbol{\lambda}_s) \tag{10}$$

where W is a weight matrix with elements in the jth row and ith column of $W_{ji} = \frac{||Q_{ij}||}{||S_i||}$. This formulation of the spatially aggregated model only uses the information that the intensities in the regions are the sums of the intersecting grid cells. The intensity for cell i is:

$$\lambda_{s,i} = \exp\left(X_s(S_i, t)\gamma + Z(S_i, t)\right) \tag{11}$$

Population density information can be added from either the region or cell level by appropriately multiplying the rows or columns, respectively.

Li et al. (2012a), building on work in Li et al. (2012b), describe a data augmentation scheme for estimating the LGCP with aggregated case data. Their approach models the counts at the intersection level. As the locations of each case within a region are not known, and so it is not known which computational grid cell to add them to, they assign a case to a grid cell within the region with probability proportional to the $w_{ij} \exp\left(Z(S_i, t)\right)$ on each iteration of the MCMC sampling and then model the intersection intensities. Despite proposing an INLA approach to modelling the random field, the method is computationally intensive given the number of intersections and the need for a data simulation step. In **rts2** we instead aim to model the counts at the region level directly and instead aggregate the intensities of the intersections, which is a more efficient method of sampling from or fitting an equivalent model. We assume that the population density and any covariates for the regions are constant throughout the area; the model is then amenable to the fitting strategies discussed in this article.

3 Maximum likelihood estimation

For maximum likelihood estimation we use the algorithm described in Watson et al. (2026). We provide a brief overview of the algorithm here. The method is a Markov Chain Monte Carlo Maximum Likelihood (MCML) algorithm, which is an expectation-maximisation type algorithm. The marginal log-likelihood of the model parameters includes an intractable integral averaging over the latent effect terms. To approximate this algorithm, MCML algorithms use Monte Carlo samples of the latent effects to generate estimates of the averages. MCML algorithms for mixed models are described by McCulloch (1997) and implemented for GLMMs in general in the **glmmrBase** package for R, which we build on for model fitting in this package. The MCML algorithm has three steps that are implemented on each iteration. On the kth iteration:

1. Sample m_k latent effects $\mathbf{z}^{(k)}$ from the distribution $\mathbf{z}|Y$ using importance sampling, with a Gaussian proposal distribution.

2. Update the estimates $\hat{\gamma}^{(k)}$ from the likelihood of Y conditional on $\mathbf{z}^{(k)}$ using a Newton-Raphson step averaging the gradient and Hessian over the latent effect samples.
3. Update the estimates of the covariance parameters $\hat{\theta}^{(k)}$ using a Newton-Raphson step averaging the gradient and Hessian over the latent effect samples.

The steps are then repeated until convergence, see below.

Stopping criteria

MCML algorithms are subject to Monte Carlo error so do not converge to a single point estimate, but reach a stationary, post-convergence phase where the log-likelihood does not improve, on average. Previously proposed stopping criteria for these algorithms, such as Caffo et al. (2005), consider a null hypothesis test that the running mean of the difference in log-likelihood values is zero. However, passing such a test may require a relatively large number of post-convergence iterations, which may be computationally burdensome for spatial models. Watson et al. (2026) propose a different stopping criterion for stochastic gradient descent algorithms for GLMMs. In particular, if the difference in successive likelihood values is:

$$\Delta\mathcal{L}(k|k-1) = \mathcal{L}^{(k)}(\hat{\gamma}^{(k)}, \hat{\theta}^{(k)}) - \mathcal{L}^{(k)}(\hat{\gamma}^{(k-1)}, \hat{\theta}^{(k-1)})$$

with μ_Δ the mean difference. Then the algorithm is halted based on the Bayes Factor:

$$BF = \frac{Pr(\mu_\Delta \leq 0|\mathcal{D}_t)}{Pr(\mu_\Delta > 0|\mathcal{D}_t)} = \frac{1-p_t}{p_t}\frac{1-\pi_t}{\pi_t}$$

where $\mathcal{D}_t$ is the values of the difference in log-likelihood values from the preceding recent iterations, p_t is the one-sided p-value from a null hypothesis test of $H_0 : \mu_\Delta > 0$ versus $H_1 : \mu_\Delta \leq 0$ and π_t, which is the prior probability $Pr(\mu_\Delta \leq 0)$ at iteration t. The prior probability is determined by a Weibull model with expected number of iterations until convergence of k_0. Thus, the stopping criterion is specified according to the threshold for BF and values of k_0 and the history length.

Region model

Further details of the model fitting algorithm are provided in Watson et al. (2026). We provide a discussion of extensions to the algorithm for spatially aggregated data here. The first step is to update the proposal distribution of the latent effects, the posterior mean of which is updated using an iteratively re-weighted least squares procedure. The random effects are sampled on a transformed scale $\boldsymbol{v} \sim N(0, I)$. Letting L be the Cholesky decomposition of $\Sigma = LL^T$, $\boldsymbol{\lambda}_r = W\boldsymbol{\lambda}_s$, $\boldsymbol{\psi}$ be a vector with ith element $y_i/\lambda_{r,i}$, then the update to the posterior mean of $\boldsymbol{v}$, $\bar{\boldsymbol{v}}$ is:

$$\begin{aligned} C &= diag(\boldsymbol{\lambda}_r^{-\frac{1}{2}})Wdiag(\boldsymbol{\lambda}_s)L \\ \boldsymbol{a} &= C^TC\bar{\boldsymbol{v}} + (Wdiag(\boldsymbol{\lambda}_s)L)^T(\boldsymbol{\psi} - \mathbf{1}) \\ \bar{\boldsymbol{v}}_{new} &= \boldsymbol{a} - C^T(CC^T + I)^{-1}C\boldsymbol{a} \end{aligned}$$

which is repeated iteratively until convergence. The computational complexity of this step is $O((rT)^3)$. To generate a Monte Carlo sample from this distribution we draw $\mathbf{v}_a, \mathbf{v}_b \sim N(0, I)$, then generate $\mathbf{v_c} = \mathbf{v}_a + C^T\mathbf{v}_b$, and $\mathbf{v}_{sample} = \mathbf{v_c} - C^T(CC^T + I)^{-1}C\mathbf{v_c} + \bar{\boldsymbol{v}}$. Importance weights are calculated for each sample w_m for $m = 1, ...M$.

The updating step for β uses a Newton-Raphson step with estimated gradient:

$$\boldsymbol{g}_\beta = \sum_{m=1}^{M} w_m X^T diag(\boldsymbol{\lambda}_s)W^T(\boldsymbol{\psi} - \mathbf{1})$$

and Hessian:

$$H_\beta = \sum_{m=1}^{M} w_m X^T diag(\boldsymbol{\lambda}_s) W^T diag(\boldsymbol{\lambda}_r^{-1}) W diag(\boldsymbol{\lambda}_s) X \tag{12}$$

so that $\hat{\beta}_{new} = \hat{\beta} + H_\beta^{-1} \boldsymbol{g}_\beta$. The updating step for the covariance parameters is the same as the standard GLMM algorithm.

Standard errors

We report the usual generalized least squares standard errors. For the region model, the fixed effect parameter standard errors are generated from the inverse of the negative of the matrix in (12).

4 Bayesian model fitting

There are several approaches to estimating the LGCP in a Bayesian framework. Letting $\Theta = [\beta, \theta, \sigma, Z]$ represent all the model parameters, the Bayesian approach aims to draw samples from the posterior

$$f_{\Theta|\mathcal{D}}(\Theta|\mathcal{D}) \propto f_{\mathcal{D}|\Theta}(\mathcal{D}|\Theta) f_\Theta(\Theta)$$

Markov Chain Monte Carlo (MCMC) methods are often used. In the context of the LGCP, there has been some discussion of the choice of the proposal density for the Metropolis-Hastings algorithm. Møller et al. (1998) propose a Langevin kernel, which is known as the Metropolis-adjusted Langevin algorithm (MALA). MALA is a special case of Hamiltonian Monte Carlo (HMC), which uses Hamiltonian dynamics to construct proposals for the Metropolis-Hastings algorithm (Carpenter et al., 2017). MALA was used by (Taylor et al., 2013) for fitting an LGCP. HMC is potentially more efficient than MALA though (Teng et al., 2017).

MCMC algorithms have the advantage of providing consistent estimation of various characteristics of the posterior distribution, including well calibrated probabilities. However, it is computationally intensive, and so may provide a further limit on the granularity of predictions in settings where time is a constraint. Several deterministic methods for approximate Bayesian inference have been proposed to try to overcome this limitation. One such method is Variational Bayes (VB) using an approximating density $r(\Theta; \phi)$ parameterised by ϕ. VB minimises the Kullback-Leibler (KL) divergence from the approximation density to the posterior:

$$\min_\phi \mathrm{KL}(r(\Theta;\phi) || f_{\Theta|\mathcal{D}}(\Theta|\mathcal{D}))$$

For the LGCP, the KL divergence lacks an analytical form, so a proxy is used, the evidence lower bound:

$$\mathcal{L}(\phi) = E\left[\log f_{\Theta,\mathcal{D}}(\Theta, \mathcal{D})\right] - E\left[\log r(\Theta;\phi)\right]$$

where the expectations are with respect to the prior densities. The minimisation problem is then to find ϕ that maximises $\mathcal{L}(\phi)$ such that the approximation density is in the support of the posterior. One of the difficulties with VB is deriving an appropriate family of approximating densities and then determining the appropriate model-specific quantities for the VB problem. Teng et al. (2017) give a specific derivation for the full LGCP. However, an alternative approach is to use automatic differentiation methods that provide a general algorithm for probabilistic models, which enables use of the Gaussian process approximations discussed here.

Calculation of $(I - A)^{-1}D^{1/2}$
  Let $\tilde{L}$ be an $n \times n$ lower triangular matrix
  for all $i \in 1, ..., n$ **do**
    for all $k \in i, ..., n$ **do**
      Let $l = A^T_{\mathcal{N}_i,i}\tilde{L}_{\mathcal{N}_i,k}$
      if $i = k$ **then**
        $\tilde{L}_{i,k} = (1 + l)\sqrt{D_{kk}}$
      else
        $\tilde{L}_{i,k} = l\sqrt{D_{kk}}$
      end if
    end for
  end for

Figure 1: Calculation of $(I - A)^{-1}D^{1/2}$

5 Gaussian process approximations

Evaluating the likelihood is computationally expensive owing to the need to invert Σ. Computational complexity and memory requirements to evaluate the log likelihood in Equation (9) scale as $O(Tn^3)$ and $O(Tn^2)$, respectively. For the Bayesian model fitting methods, these models therefore quickly become infeasible with even moderately sized grids, which can limit their use. We offer an option to skip model fitting with "known" covariance parameter values, which can be seen as akin to the problem of choosing a bandwidth for kernel-based approaches. We provide a function to estimate the empirical semivariogram to support parameter choice. However, for Bayesian fitting procedures including the covariance parameters, there are two main classes of approximation to the likelihood of a Gaussian process we include.

Nearest neighbour gaussian process

The multivariate Gaussian likelihood can be rewritten as the product of the conditional densities:

$$f(z) = f(z_1)\prod_{j=2}^{n} f(z_j|z_1, ..., z_{j-1})$$

where, for convenience, z here represents a spatial-only field. Vecchia (1988) proposed that one can approximate $f(z)$ by limiting the conditioning sets for each z_j to a maximum size of m. For geospatial applications, Datta et al. (2016a,b) proposed the nearest neighbour Gaussian process (NNGP), in which the conditioning sets are limited to the m 'nearest neighbours' of each observation.

Let $\mathcal{N}_j$ be the set of up to m nearest neighbours of j with index less than j. We use the notation $\Sigma_{\mathcal{N},\mathcal{M}}$ to represent the submatrix of Σ with rows in $\mathcal{N}$ and columns in $\mathcal{M}$, where these sets could be of size one. The approximation is:

$$f(z) \approx f(z_1)\prod_{j=2}^{n} f(z_j|\mathcal{N}_j)$$

which leads to:

$$\begin{aligned} z_1 &= \eta_1 \\ z_j &= \sum_{i=1}^{m} a_{ji} z_{\mathcal{N}_{ji}} \end{aligned} \tag{13}$$

where $\mathcal{N}_{ji}$ is the ith nearest neighbour of j.

Equation (13) can be more compactly written as $z = Az + \eta$ where A is a sparse, strictly lower triangular matrix, $\eta \sim N(0, D)$ with D a diagonal matrix with entries $D_{11} = \text{Var}(z_1)$

and $D_{ii} = \text{Var}(w_i|\mathcal{N}_i)$ (Finley et al., 2019). The approximate covariance matrix can then be written as $\Sigma_0 \approx (I - A)^{-1}D(I - A)^{-T}$ such that the quadratic form is approximated as:

$$\sum_t \mathbf{z}_{[t]}(I - A)^T D^{-1}(I - A)\tilde{\mathbf{v}}_{[t]}$$

As $I - A$ is upper triangular, the quadratic form can be calculated cheaply using forward substitution, iterating only over the nearest neighbours for each row. The log determinant is:

$$\log(|\Sigma_0|) \approx \sum_{i=1}^{m} -\log(D_{ii})$$

Finley et al. (2019) provide algorithms for the calculation of the matrices A and D. The non-zero elements of A are given by the linear system $\Sigma_{0;\mathcal{N}_j,\mathcal{N}_j} A^T_{j,\mathcal{N}_j} = \Sigma_{0;\mathcal{N}_j,j}$, where the subscripts indicate the indices of the relevant submatrix, and the diagonal elements of matrix D are $D_{j,j} = \Sigma_{0;j,j} - A_{j,\mathcal{N}_j}\Sigma_{0;\mathcal{N}_j,j}$. The NNGP has computational complexity scaling with $O(Tnm^3)$ and so the log-likelihood is now linear in both time and grid size.

There are two main factors that can affect the accuracy of the NNGP approximation. The ordering of the grid cells in terms of their indices can affect performance of the approximation. Guinness (2018) compares the performance of several different ordering schemes. They suggest that a "minimax" scheme, in which the next observation in the order is the one which maximises the minimum distance to the previous observations, often performed best, although in several scenarios ordering in terms of the vertical coordinate, or at random, provided comparable performance. The number of neighbours did not appear to affect the relative performance of the orderings; they considered both 30 and 60, although Datta et al. (2016a) uses 15 nearest neighbours in their simulations.

Hilbert space gaussian process

Low, or reduced, rank approximations aim to approximate the matrix Σ with a matrix $\tilde{\Sigma}$ with rank $m < n$. The optimal low-rank approximation is $\tilde{\Sigma} = \Phi\Lambda\Phi^T$ where Λ is a diagonal matrix of the m leading eigenvalues of Σ and Φ the matrix of the corresponding eigenvectors. However, the computational complexity of generating the eigendecomposition scales the same as matrix inversion. Solin and Särkkä (2020) propose an efficient method to approximate the eigenvalues and eigenvectors using Hilbert space methods, so we refer to it as a Hilbert Space Gaussian Process (HSGP). Riutort-Mayol et al. (2023) provide further discussion of these methods.

Stationary covariance functions, including those in the Matern class like exponential and squared exponential, can be represented in terms of their spectral densities. For example, the spectral density function of the squared exponential function in d dimensions is:

$$Sp(\omega) = \sigma^2(\sqrt{2\pi})^d \phi^d \exp(-\phi^2\omega^2/2)$$

Consider first a unidimensional space ($d = 1$) with support on $[-c, c]$. The eigenvalues λ_j (which are the diagonal elements of Λ) and eigenvectors ϕ_j (which form the columns of Φ) of the Laplacian operator in this domain are:

$$\lambda_j = \left(\frac{j\pi}{2c}\right)^2$$

and

$$\phi_j(s) = \sqrt{\frac{1}{c}}\sin\left(\sqrt{\lambda_j}(x + c)\right)$$

Then the approximation in one dimension is

$$\mathcal{Z}(s) \approx \sum_{j=1}^{m} Sp\left(\sqrt{\lambda_j}\right)^{1/2} \phi_j(s)\beta_j$$

where $\beta_j \sim N(0,1)$. This result can be generalised to multiple dimensions. The total number of eigenvalues and eigenfunctions in multiple dimensions is the combination of all univariate eigenvalues and eigenfunctions over all dimensions. For example, if there were two dimensions and $m = 2$ then there would be four multivariate eigenfunctions and eigenvalues equal to the four combinations over the two dimensions. The approximation is then as described above but with the multivariate equivalents.

The approximation can be used in the full model such that the intensity becomes

$$\lambda(S_i,t) = r(S_i,t)\exp\left(X(S_i,t)\gamma + (P^{\frac{1}{2}} \otimes \Phi\Lambda^{\frac{1}{2}})\beta)\right) \tag{14}$$

and where $\beta \sim N(0, I_{m^2})$. The matrix Φ does not depend on the covariance parameters and can be pre-computed, so only the product $\Phi\Lambda^{\frac{1}{2}}$ needs to be re-calculated during model fitting, which scales as $O(Tnm^2)$.

The performance of both the HSGP and the NNGP depends on m, either the number of basis functions or the number of nearest neighbours, respectively, and several other parameters. The other key value for the HSGP is the boundary condition c. Riutort-Mayol et al. (2023) provide a detailed analysis of the HSGP for unidimensional linear Gaussian models, examining in particular how the choice of c and m affect performance of posterior inferences and model predictions. Here, we assume c is relative to the maximum and minimum coordinate in each dimension (i.e. the space is scaled to $[-1,1]^2$). They find values of $m = 15$ and $c = 1.5$ to 2.5 provide a good approximation for the models they consider with squared exponential covariance function, but also show that for higher values of m we generally require larger values of c. For example, $m = 20$ and $c = 2$ provides similar performance to $m = 30$ and $c = 4$.

6 Related software and methods

Software packages for fitting an LGCP are relatively few. In the R ecosystem, we are aware of two dedicated packages. First, **lgcp**(Taylor et al., 2013, 2015) provides maximum likelihood and Bayesian MCMC estimation of the LGCP model. However, it uses only the full model and while there are some features to improve computational time, it suffers from long running times for all but the smallest applications. The temporal structure of the implemented model does not scale linearly with time either. Furthermore, it makes use of a no-longer supported package and standard for spatial data (**sp**) and does not make use of state-of-the-art Bayesian methods.

The R package **inlabru** (Bachl et al., 2019) provides a set of tools to approximate Bayesian posteriors for the LGCP using integrated nested Laplacian approximation (INLA), a popular and highly efficient Bayesian approximation. INLA (made available in R more generally through **R-INLA** (Lindgren and Rue, 2015)) approximates the posterior means of model parameters, and in this case spatially-continuous Gaussian processes, using Gaussian Markov random fields (GMRFs). A comprehensive overview is provided in Rue and Held (2005) Chapter 5. We do not include INLA given the availability of **inlabru**. The quality of INLA in terms of quantifying uncertainty beyond posterior means may also be less than needed for surveillance applications. Taylor and Diggle (2014) compare MCMC (specifically MALA) and INLA using the methods in **lgcp** and **inlabru** and conclude that while MCMC does take much longer, it offers much better predictive accuracy. However, the scale of the problems examined may be considered relatively small; for larger grids and multiple time periods, MCMC may be prohibitively slow. Teng et al. (2017) come to similar conclusions in a broader comparison of MCMC and other methods.

More recently, the package **stelfi** (Jones-Todd and van Helsdingen, 2024) has been released that provides maximum likelihood estimation and predictions from spatio-temporal LGCPs and Hawkes processes. The package provides novel functionality for temporal and spatiotemporal self-exciting point process models, which we do not currently include in **rts2**. Dovers et al. (2024) describes how generic generalised additive model fitting software, particularly **mgcv** in R, can be used to fit LGCPs as the software includes a basis function expansion to approximate the Gaussian random field. We include a low-rank approximation in this package.

Finally, we note two approaches in the literature to modelling spatially aggregated case data. (Johnson et al., 2019) describes an approach for fitting and predicting from an LGCP model with spatially aggregated data using an MCMC sampling scheme and assuming a piecewise constant latent field in each area. At the time of writing though, software to implement this approach was no longer available on CRAN. Li et al. (2012a) describe a Bayesian data augmentation approach to fitting these models, which can be implemented in general Bayesian software.

We are not aware of any packages specifically designed for LGCPs, or real-time surveillance more generally, for other software environments like Stata or SAS. However, one may also fit LGCPs using more general mixed model software. For example, provided one generate the appropriate grid data, the R package **glmmTMB** offers Laplace approximation model fitting for GLMMs including Poisson models with spatial covariance structure. One can extract the processed grid data generated by **rts2** for use in other packages. For full maximum likelihood model fitting one can use **glmmrBase**. For Bayesian MCMC the Stan probabilistic programming language can fit any of the models described here. The **rts2** package uses **glmmrBase** and **Stan** for model fitting. In Stata and SAS, one could theoretically also use the GLMM fitting functionality to fit these models; however, as with the other methods, they would be impractically slow for even moderately sized grids. We are not aware of any specific implementation of LGCP models in other software environments. We are also not aware of any other implementation of what we have termed the 'region model' or of NNGP or HSGP with LGCPs.

7 rts2 package

We now describe the functioning of the package in R. The accompanying R and data files provide reproducible code for all these examples, including pre-estimated models. We provide some comparisons between the different methods. MCMC sampling uses Stan through **rstan** (Carpenter et al., 2017).

The main functionality of **rts2** is provided by the `grid` class, which uses the **R6** package's object-orientated class system. An object-orientated method is preferred here, as much of the functionality of the package involves repeatedly manipulating the same data object. As we demonstrate below, use of an object-orientated class system simplifies the syntax relative to the more standard R functional methods in these cases. Geographic data uses the **sf** package. Initialisation of a new `grid` object can use either a single polygon describing the boundary of an area of interest, or a set of polygons in the case of the region data model described above.

7.1 Point data

We use simulated data for this first example, which are provided in the package, as real-world location-identifying data are typically not anonymised. These data, along with the relevant spatial data objects, are provided in the replication materials. The boundary is also provided and represents the city of Birmingham, UK along with a set of covariate data at the middle layer super output area (MSOA) level, which is the second smallest census tract area.

To instantiate a new `grid` object we require the boundary and the cell size of the grid:

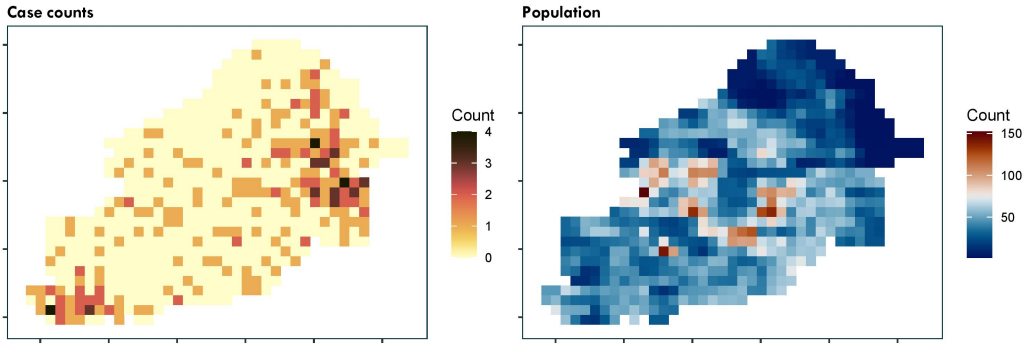

Figure 2: Example simulated data for the city of Birmingham, UK. Left panel: case counts of point data mapped to the computational grid. Right panel: population density mapped to the computational grid.

```
g1 <- grid$new(boundary,cellsize=0.008)
```

The next step is to map the point data to the grid to create the counts. We provide the convenience function `create_points()` to generate **sf** point data from an ordinary R data frame with location and possibly time data. Then, the member function `points_to_grid()` will aggregate the case counts, for example:

```
g1$points_to_grid(point_data = create_points(y,
                                             pos_vars = c("Y","X"),
                                             t_var = c("t")),
                  t_win = "month",
                  laglength = 1)
```

or without a temporal component:

```
g1$points_to_grid(point_data = create_points(y, pos_vars = c("Y","X")))
```

For spatio-temporal analysis, we could determine here what size time step we want (day, week, or month) and how many previous periods to include (`laglength`). The choice of lag length is similar to the choice of cell size; we want it to be long enough to provide the best predictions of current incidence, but as short as possible otherwise to minimise the required computational resources. The function adds columns to the grid object (labelled `t*` for spatio-temporal or `y` for spatial-only). Figure 2 top-left panel shows the aggregated case counts, which can be plotted using `g1$plot("y")`.

7.2 Overlaying covariate data

We can include covariates in our model. The aim of including covariates in surveillance applications is not typically to provide inference about the effect of any particular covariate but to improve predictions or to generate conditional, e.g. age-adjusted, predictions of relative risk. For example, by including day of the week as a covariate we can capture differences in hospital attendances and admissions resulting from individual behaviour not related to variations in disease incidence. At a minimum we would suggest including population density to enable population-standardised predictions. Covariates can be added to the grid data using the member function `add_covariates()`.

The covariate data are provided to the function in a form that can be manipulated into polygons, either **sf** or **raster** data. There are several approaches to mapping data onto the grid provided in the package. Primarily, we offer a straightforward simple 'flat' mapping of the covariate data. Let W_c be an $n_c \times n$ weight matrix, where n_c is the number of polygons in the covariate data, and the entries of W_c are equal to the areas of the intersections of each

grid cell and covariate polygon. The mapping is $t(W_c)\mathbf{x}_c/\mathbf{w_c}$ where $\mathbf{x}_c$ is the covariate data for each polygon, and $\mathbf{w_c}$ are the column sums of W_c.

Spatially-varying only covariates

Where the time step is short, such as days, it is unlikely there will be data that varies by time step over the area of interest. For example, population density data will be static over the area of interest. To add spatially-varying covariates to the grid data we require an `sf` object with polygons encoding the covariate data. The `add_covariates()` function then, for each grid cell, takes an average of the values in the overlapping polygons, weighted by either the size of the area of overlap or the population, if population density data are available. A good source of population density predictions is WorldPop (at www.worldpop.org), which will provide rasters at 100m or 1km resolution for all countries.

First, we add the population density:

```
g1$add_covariates(msoa,
                  zcols="pop",
                  weight_type="area")
```

where `msoa` is the MSOA data as an `sf` object. Figure 2 top right panel shows the population density.

Temporally-varying only covariates

The example includes only a single period, but in other cases with multiple time periods we may have both temporally and spatio-temporally varying covariates. Some covariates, like day of the week, vary over time but are the same for all grid cells in each time period. To add a temporally-varying covariate we need to produce a data frame with a column for time period (from 1 to `laglength`) and then columns with the values of the covariates. To obtain day of the week we can use the member function `get_dow()`, which will return a data frame mapping the date of each time period to a day of the week. For example,

```
df_dow <- grid_data$get_dow()
print(df_dow)
```

will return the data frame and then these columns can then be added to the main grid data:

```
grid_data$add_covariates(df_dow,
                         zcols=c("dayMon","dayTue",
                                 "dayWed","dayThu","dayFri",
                                 "daySat","daySun"))
```

The member function `add_time_indicators()` will generate fixed effect indicator variables for each time period in multi-period data. These will be labelled `time1i`, `time2i`, and so forth.

Spatially- and temporally-varying covariates

In some cases there may be covariates that vary over time and space. There are two ways of adding these into the main grid data. Both ways need to generate, for each covariate, as many columns as there are time periods. The column names will be the covariate name followed by the time period index.

If the covariate data are included in different data frames for each time period, then we can repeat the process above for each time period and add a label using the `t_label` argument:

```
grid_data$add_covariates(msoa_t1,
                         zcols=c("covA","covB",...),
                         t_label = 1)
```

Alternatively, if the value of the covariates are all included in different columns of the same data frame then they can be added at the same time, ensuring we keep to the same naming convention of covariate name and time index as a string:

```
grid_data$add_covariates(msoa,
                         zcols=c("covA1","covA2","covA3",...))
```

7.3 Bayesian priors

For the Bayesian models, one must consider the prior distributions of the parameters. Currently, these distributions are proscriptive and one must only select the location and scale. For the lengthscale ϕ and variance (σ^2) parameters, we specify half-normal distributions (i.e. a normal distribution truncated below zero). And for the parameters of the linear prediction γ, we specify normal distributions. The choice of mean and standard deviation values for these prior distributions should be provided to the `grid` object as a list:

```
grid_data$priors <- list(
  prior_lscale=c(0,0.5),
  prior_var=c(0,0.5),
  prior_linpred_mean=c(-5,rep(0,7)),
  prior_linpred_sd=c(3,rep(1,7))
)
```

7.4 Model fitting

The two model fitting functions are `lgcp_bayes()` and `lgcp_ml()` for Bayesian and maximum likelihood methods, respectively. The arguments to the two functions are highly similar. The functions described above add the case counts and covariates to the grid data. The model fitting functions require specification of which covariates to include, the model to use, and any approximation and fitting parameters. For example, to use the exponential covariance function with the NNGP approximation with ten nearest neighbours, we can use the following code. We first re-order the computational grid using the 'minimax' ordering.

```
g1$lgcp_ml(popdens = "pop")
```

The model fit can be returned from this call, but it is also stored internally and can be returned using `g1$model_fit()`, which will return an S3 object of class `rtsFit`.

Figure 3 shows the predicted relative risk from three different model fitting methods. We discuss extraction and plotting of model fits in the next section. Code to generate these model fits is provided in the replication materials. The predictions are qualitatively similar, although there are notable differences in the smoothness and magnitude of the relative risk surface, which result from different choices about approximation parameters and other aspects of the analysis.

7.5 Region data

To illustrate the use of the regional data model we use publicly available crime data from the UK. Monthly counts of different types of crime are available at the middle-layer super output area level from crime.uk. We extract the monthly counts for 2022 for the city of Birmingham, UK for burglary. These data are available in the package.

If a new `grid` class object is instantiated with data comprised of multiple polygons, it assumes a regional data model. The provided polygons are assumed to represent all the areas of interest. Generally, these should be a division of a contiguous area, although this is not a strict requirement. Since the context for a regional data model is that the data are only available in their aggregated format, the data used to initialise the new `grid` object should already contain these counts. Column names should be in the same format as generated for point data described above: if there is only a single time period then the counts should be in a column named `y`, otherwise the columns should be labelled `t1`, `t2`, `t3`, ... and so

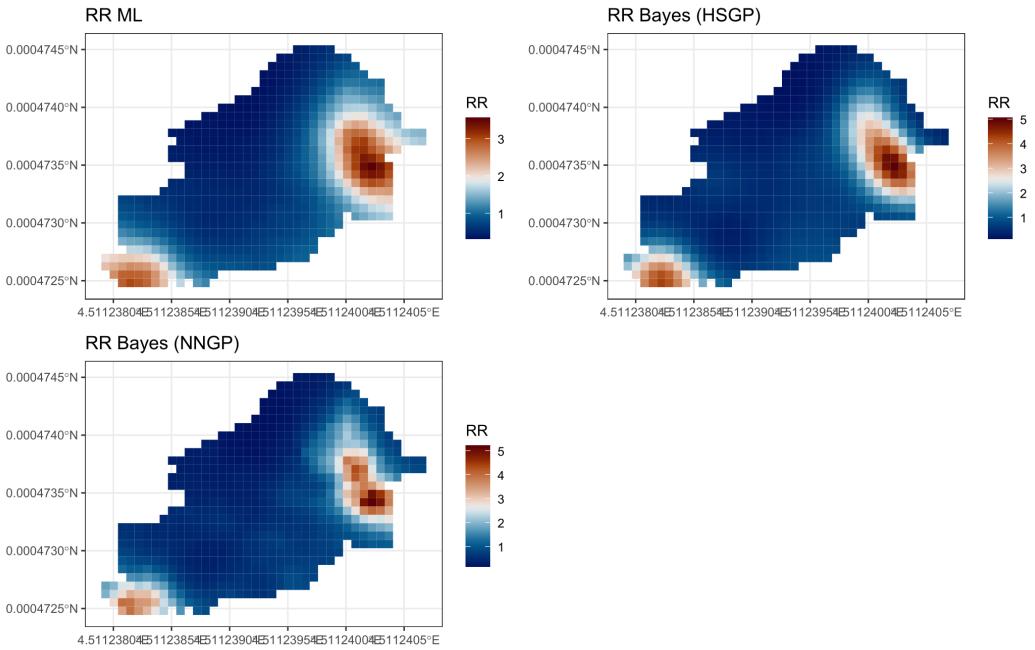

Figure 3: Example model fits using three methods: maximum likelihood (ML) and MCMC along with HSGP and NNGP approximations. The HSGP model fits used 15 basis function per dimension and the NNGP used 25 nearest neighbours.

forth in chronological order. As with the point data we must also provide a cell size for the computational grid. Covariates must also be included in the regional data object with the same naming conventions for multi-period models. One can add grid level covariates from another source using the `add_covariates()` function as described above.

```
g2 <- grid$new(msoa,cellsize = 0.008) # create new region grid
g2$add_time_indicators()
```

We do not include any additional covariates in this example beyond the time indicators. Model fitting uses the same commands as for the basic spatial data model described above. We fit the model with maximum likelihood and both NNGP and HSGP approximations:

```
g2$lgcp_ml("pop", # population density
           covs = paste0("time",2:12,"i") # time period indicators)
```

7.6 Analysing the output

Figure 4 shows several outputs from the models fitted in the previous section. Here, we describe how to extract these outputs.

Predictions

The output from the model enables us to generate four main predictions across our area of interest:

1. Predicted total incidence.
2. Predicted incidence per head of population.
3. The relative risk, i.e. the relative difference between predicted and expected incidence. For example, a relative risk of 2 would imply that the predicted incidence is twice as high as would be expected given the area mean and local covariate values.

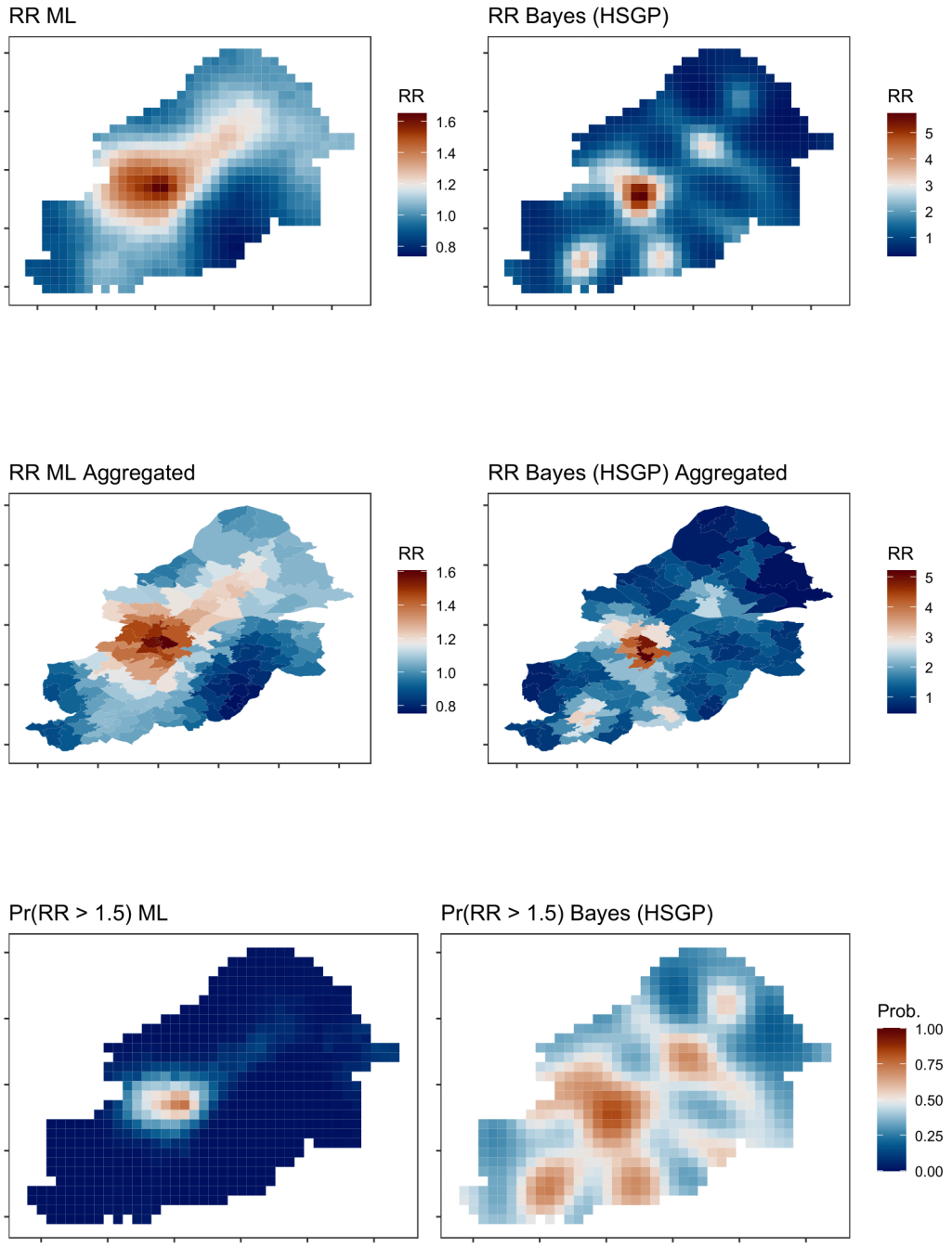

Figure 4: Model fitting outputs for the Birmingham crime example. The left column provides outputs from a maximum likelihood model fit; the right column is an HSGP Bayesian fit with 10 basis functions per dimension. The top row is the predicted relative risk, the middle row the aggregated relative risk, and the bottom row the probability that the relative risk is greater than 1.5.

4. The incidence rate ratio relative to some previous time point for spatio-temporal analyses. For example, we may be interested in comparing incidence to seven days prior; a value of 1.5 would imply that the incidence has risen 50% in a week.

We can extract any or all of these predictions using the function `extract_preds()`, which will extract the predictions from the last model fit. For this example, we will extract from the crime data analysis the relative risk and incidence rate ratio. For region data models incidence predictions and incidence rate ratio are added to the region data, with the relative risk added to the grid data.

```
g2$extract_preds(type = c("rr","irr"),
                 irr.lag = 11,
                 popdens = "pop")
```

Hotspots

The concept of a "hotspot" is often poorly or vaguely defined and it can mean different things to different people (Lessler et al., 2017). Generally, we can define it as an area where there is a high probability that some epidemiological measure(s) exceed predefined threshold(s). Here, we can use any of the outputs of the model listed above (incidence, relative risk, or incidence rate ratio). The `hotspots()` member function allows us to define a hotspot based on any of these criteria. For example, we can calculate the probability that the relative risk exceeds 1.5 across the grid:

```
g2$hotspots(threshold = 1.5, stat = "rr", popdens = "pop")
```

will add a column to the grid data labelled `rr` with the probabilities that the cell has a relative risk greater than 1.5.

7.7 Aggregating to other geographies

It may be desirable to summarise the results of the analysis on a more familiar geography or to aggregate grid data to the region data. For example, we may wish to aggregate the results onto administrative areas that might be used to direct public health responses. In this example, we aggregate the relative risk results back onto the MSOA geography for the city. As with the covariate averaging we can either average with respect to the area or the population, here we'll average with respect to the area.

```
g2$region_data <- g2$aggregate_output(g2$region_data, zcols=c("rr"))
```

The second row of Figure 4 shows an example.

7.8 Visualisation

As the output of the analysis is an `sf` object (stored in the slot `g2$grid_data`), there are many different tools we can use to plot and visualise the results. We can use the default plot function in the `grid` class, which calls `sf`'s plot function for a variable in the grid data. For example, for the relative risk:

```
grid_data$plot("rr")
```

Other plotting tools include **ggplot2** with the `geom_sf` function; **tmap** provides a range of tools for map plotting including allowing multiple panels to be plotted. The `mapview` package provides functionality to create interactive overlays of `sf` data on OpenStreetMap maps.

7.9 Model parameters

We may also be interested in the posterior samples of particular model parameters, which are contained in the output of the model fit. We note two important considerations for interpreting these parameters:

- For HSGP model fitting, the coordinates are scaled to $[-1, 1]$, so the length scale parameters will be equivalently scaled. Use the `scale_conversion_factor()` function to extract these factors.
- Spatially-varying parameters of the linear predictor (γ) will differ from a non-spatial model. The parameters on spatially varying covariates will give an indication of the direction of an effect, but their magnitude may have different interpretations (Hodges and Reich, 2010; Reich et al., 2006).

8 Examples comparing packages

There are few software alternatives for some of the functionality we have described in this article. Here, we provide examples to compare outputs and code between packages.

8.1 Spatial point data

We firstly illustrate the use of the package alongside **inlabru** (Bachl et al., 2019), which provides approximate Bayesian model fitting using INLA. We will use the 'gorillas' dataset from that package, which describes the nesting sites of gorillas in an area.

inlabru model fitting

The example data already includes the meshing and preparation of covariates required for model fitting. To load the data we specify

```
require(inlabru)
data <- gorillas_sf
data$gcov <- gorillas_sf_gcov()
elev <- data$gcov$elevation
data$gcov$elev <- (elev - mean(terra::values(elev), na.rm = TRUE))/sd(terra::
    values(elev), na.rm = TRUE)
```

The elevation data is mean standardised to improve the stability of model fitting. We set the SPDE prior as following the example provided in `inlabru`. We include an intercept and standardised elevation as a covariate in the model, in addition to the spatial field. To specify the formula, fit the model, and generate predictions we use:

```
ecomp <- geometry ~ elev(f.elev(.data.), model = "linear") +
  mySmooth(geometry, model = matern) + Intercept(1)

efit <- lgcp(ecomp, data$nests, samplers = data$boundary, domain = list(geometry =
     data$mesh))
pred.df <- fm_pixels(data$mesh, mask = data$boundary)
e.pred <- predict(
  efit,
  pred.df,
  ~ list(
    int = exp(mySmooth + elev + Intercept),
    int.log = mySmooth + elev + Intercept,
    rr = exp(mySmooth)
  )
)
```

rts2 model fitting

To fit an equivalent LGCP model, using the HSGP approximation with Bayesian MCMC model fitting, in **rts2**, we follow the steps described above:

```
mod <- grid$new(
  data$boundary,
  cellsize = 0.025
)
```

```
mod$points_to_grid(data$nests)
mod$add_covariates(cov_data = data$gcov$elev,zcols = "elev")
fit3 <- mod$lgcp_ml(covs = c("elev"))
mod$extract_preds(c("pred","rr"))
```

The code will generate a grid with 3,663 grid cells, and we have specified to use 15 basis functions per dimension.

Comparison of inlabru and rts2 results

Figure 5 shows the predicted intensity surface and relative risk, and their log standard deviations, from **inlabru** and **rts2**. Qualitatively the intensity and relative risk surfaces are similar with **inlabru** showing a higher degree of smoothing. The intensity reported by **rts2** is per grid cell, whereas for **inlabru** the intensity is over the whole area. The range of the relative risk was larger with **inlabru**.

8.2 Spatially-aggregated case data

For spatially aggregated case data, a common model is a conditionally autoregressive model in which the counts in each area are independent of one another conditional on the counts in neighbouring areas. The Besag-York-Mollie (BYM) model is the key example of this approach. In R, the INLA package (van Niekerk et al., 2021) is often used for implementation of this model with spatially aggregated case data. Here, we provide a comparison of generating predictions from spatio-temporal count data using the LGCP model described above in rts2, with the version of the BYM model described by Simpson et al. (2017) implemented in INLA. We use the common exemplar dataset describing sudden infant death syndrome cases in North Carolina in the 1970s described in Cressie and Chan (1989) and elsewhere. These data are available in the `spData` package on CRAN and can be loaded using:

```
nc.sids <- sf::st_read(system.file("shapes/sids.gpkg", package="spData")[1])
```

Complete code for this example is provided in the reproduction materials.

Code for INLA

We adapt code for these data described in Palmí-Perales et al. (2021). There are several approaches to specifying the statistical model, for example, one could include additional iid noise beyond the method we specify here. This example is therefore intended to be illustrative. The processing of the data is achieved using the following code:

```
# First time period
# Compute expected cases
r74 <- sum(nc.sids$SID74) / sum(nc.sids$BIR74)
nc.sids$EXP74 <- r74 * nc.sids$BIR74

# Second time period
# Compute expected cases
r79 <- sum(nc.sids$SID79) / sum(nc.sids$BIR79)
nc.sids$EXP79 <- r79 * nc.sids$BIR79

d <- data.frame(OBS = c(nc.sids$SID74, nc.sids$SID79),
                PERIOD = c(rep("74", 100), rep("79", 100)),
                EXP = c(nc.sids$EXP74, nc.sids$EXP79))

# County-period index
d$idarea <- rep(1:100,2)

# prepare the graph
nb <- spdep::poly2nb(nc.sids)
spdep::nb2INLA("map.adj",nb)
```

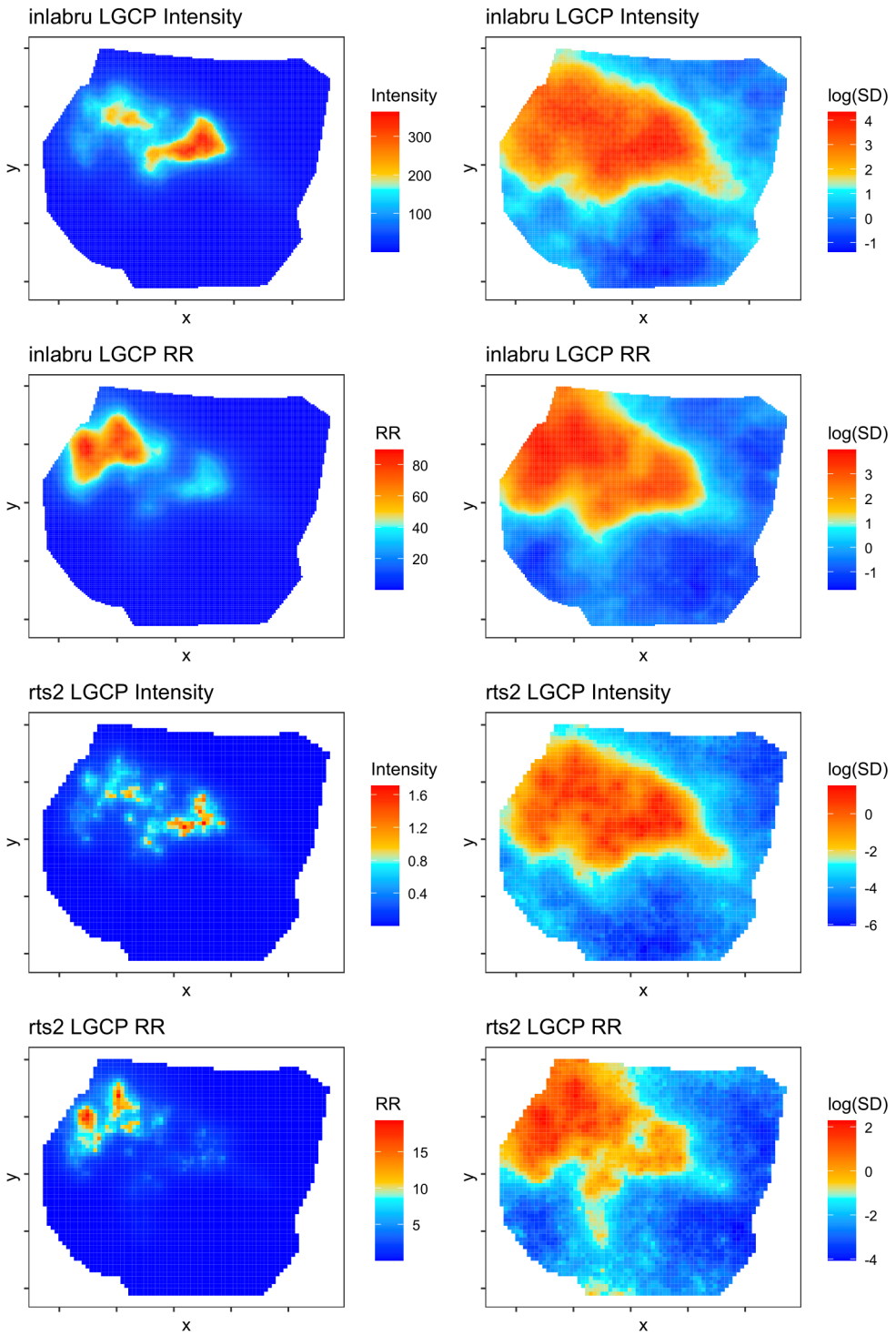

Figure 5: Predicted intensity, relative risk, and log standard deviations using the gorilla nesting sites data in **inlabru** package comparing **rts2** maximum likelihood and the SPDE approach in **inlabru**

```
require(INLA)

g <- inla.read.graph(filename = "map.adj")
```

The command in INLA to fit the model with an intercept and time period indicator, along with the BYM correlation structure is:

```
res <- inla(OBS ~ f(idarea, model = "bym2", graph = g) +  PERIOD, family = "
    poisson", data = d, E = EXP, control.predictor = list(compute = TRUE))

nc.sids$rr_inla <- res$summary.random$idarea[, "mean"][101:200]
nc.sids$rr_lci_inla <- res$summary.random$idarea[, "0.025quant"][101:200]
nc.sids$rr_uci_inla <- res$summary.random$idarea[, "0.975quant"][101:200]
nc.sids$prob_inla <- 1 - pnorm(log(1.5), nc.sids$rr_inla, res$summary.
    random$idarea[, "sd"][101:200])
nc.sids$pred_inla <- res$summary.fitted.values[, "mean"][101:200]
```

where we have used default non-informative priors.

Code for rts2

For the rts2 package, we need to generate a new `grid` object using the data `nc.sids` described above, and rename the variables relative to time period.

```
require(rts2)

mod <- grid$new(nc.sids, cellsize = 0.25)
# rename the columns
colnames(mod$region_data)[c(10,13)] <- c("t1","t2")
colnames(mod$region_data)[c(24,26)] <- c("exp1","exp2")
# add a time indicator
mod$add_time_indicators()
```

Then the model fitting and extraction of predictions are achieved using the following code. In this example, we use a maximum likelihood approach to model fitting. We again use the HSGP with the default of ten basis functions per dimension.

```
# fit the model
res3 <- mod$lgcp_ml("exp", model = "fexp",
                    covs = c("time2i"),
                    iter_sampling = 250)

mod$extract_preds(c("pred","rr"))
mod$hotspots(threshold = 1.5, stat = "rr")
# generate a fitted value net of offset
# mod$grid_data$pred_fitted <- mod$grid_data$pred_mean_pp/mod$grid_data$exp2
# generate confidence intervals for prediction using the standard errors
mod$region_data$pred_lci <- mod$region_data$pred_mean_total -
  qnorm(0.975)*mod$region_data$pred_mean_total_se
mod$region_data$pred_lci <- ifelse(mod$region_data$pred_lci >= 0,
    mod$region_data$pred_lci, 0)
mod$region_data$pred_uci <- mod$region_data$pred_mean_total +
  qnorm(0.975)*mod$region_data$pred_mean_total_se

# attach to the original data file
nc.sids$pred <- mod$region_data$pred_mean_total
nc.sids$pred_lci <- mod$region_data$pred_lci
nc.sids$pred_uci <- mod$region_data$pred_uci
```

Comparison of outputs

Figure 6 shows the predicted relative risk and standardised mortality ratios at the county level using **INLA** and **rts2**. An evident difference is the degree of smoothing is lower in **rts2**. The magnitude of the relative risk is comparable between the two approaches, and the broad spatial patterns are comparable. However, the specific counties identified as high probability of an increased relative risk differ.

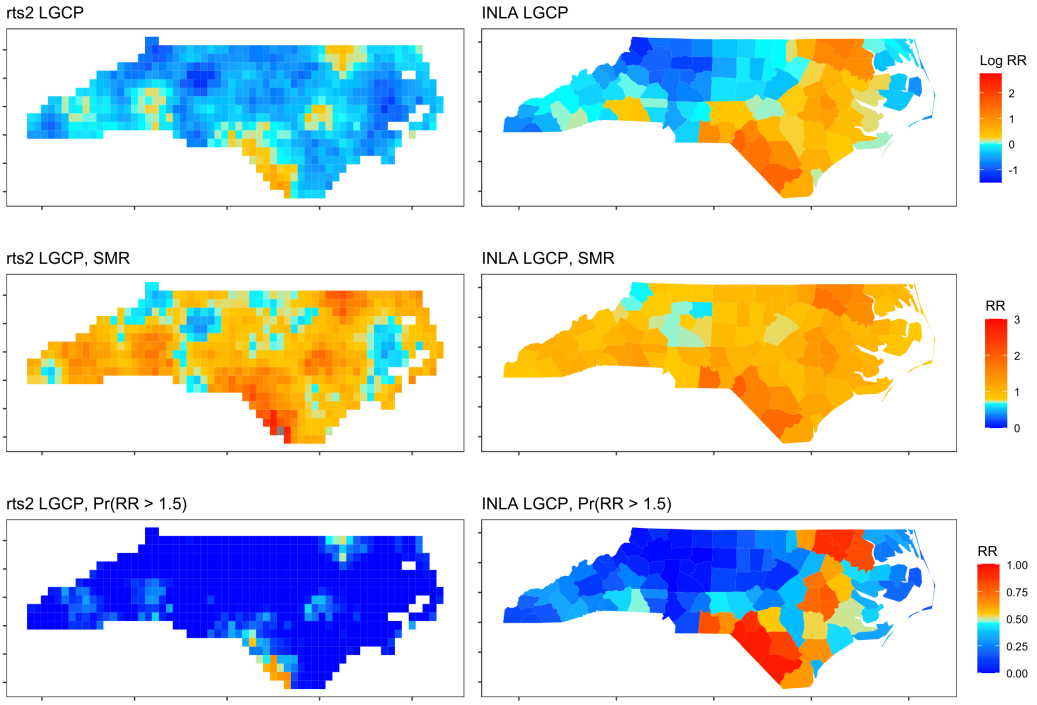

Figure 6: Predictions of the relative risk, standardised mortality ratio, and probability RR > 1.5 of SIDS in counties in North Carolina in 1979 using rts2 and INLA

9 Conclusions

The **rts2** package provides multiple different methods and approximations for fitting the LGCP. The motivation for this package was to support real-time disease surveillance efforts by providing predictions in a fraction of the time of a full LGCP model with no approximation, while maintaining the reliability and calibration of the quantification of uncertainty. In a set of comparisons, models that took days to run can now be run in minutes or hours, using either maximum likelihood or Bayesian methods with approximation. Watson et al. (2026) illustrate that significant reductions in computational time can also be achieved using modern GPU hardware, which is a future objective with these models. We also included Bayesian methods with approximations that can be 'tuned': at the extreme the approximations are equal to the full model. However, further research is required to establish the quality of the predictions and calibration of uncertainty, especially when compared with other 'fast' alternatives like INLA and kernel-based methods. We have also provided functionality for data manipulation and extraction from model fits to support adoption and use of these methods.

The package provides a variety of model fitting methods and approaches, including both Bayesian and maximum likelihood methods, and different Gaussian Process approximations. Given the variety of methods, and the existence of other alternatives including INLA, a full simulation-based comparison is warranted to determine the quality of predictions and parameter estimation. In terms of computational time, the HSGP method is the fastest and scales well with increasing number of basis functions. We have also found qualitatively that the method is generally stable and produces results comparable with similar methods, as demonstrated by our examples.

While our aim with this package is to support 'real-time disease surveillance' the methods are applicable to multiple areas where case data are used to identify 'hotspots' to support public and policy responses. We used an example with crime data. Other areas include geology, ecology, social policy, and many more.

References

F. E. Bachl, F. Lindgren, D. L. Borchers, and J. B. Illian. inlabru: an R package for Bayesian spatial modelling from ecological survey data. *Methods in Ecology and Evolution*, 10: 760–766, 6 2019. ISSN 2041-210X. doi: 10.1111/2041-210X.13168. [p152, 160]

B. S. Caffo, W. Jank, and G. L. Jones. Ascent-based Monte Carlo expectation– maximization. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 67:235–251, 4 2005. ISSN 1369-7412. doi: 10.1111/j.1467-9868.2005.00499.x. [p148]

B. Carpenter, A. Gelman, M. D. Hoffman, D. Lee, B. Goodrich, M. Betancourt, M. Brubaker, J. Guo, P. Li, and A. Riddell. Stan : A probabilistic programming language. *Journal of Statistical Software*, 76, 2017. ISSN 1548-7660. doi: 10.18637/jss.v076.i01. [p149, 153]

N. Cressie and N. H. Chan. Spatial modeling of regional variables. *Journal of the American Statistical Association*, 84:393–401, 6 1989. ISSN 0162-1459. doi: 10.1080/01621459.1989.10478783. [p161]

A. Datta, S. Banerjee, A. O. Finley, and A. E. Gelfand. Hierarchical nearest-neighbor Gaussian process models for large geostatistical datasets. *Journal of the American Statistical Association*, 111:800–812, 4 2016a. ISSN 0162-1459. doi: 10.1080/01621459.2015.1044091. [p150, 151]

A. Datta, S. Banerjee, A. O. Finley, and A. E. Gelfand. On nearest-neighbor Gaussian process models for massive spatial data. *WIREs Computational Statistics*, 8:162–171, 9 2016b. ISSN 1939-5108. doi: 10.1002/wics.1383. [p150]

P. Diggle, B. Rowlingson, and T.-l. Su. Point process methodology for on-line spatio-temporal disease surveillance. *Environmetrics*, 16(5):423–434, aug 2005. ISSN 1180-4009. doi: 10.1002/env.712. URL http://doi.wiley.com/10.1002/env.712. [p144]

P. J. Diggle and E. Giorgi. Model-Based Geostatistics for Prevalence Mapping in Low-Resource Settings. *Journal of the American Statistical Association*, 111(515):1096–1120, jul 2016. ISSN 0162-1459. doi: 10.1080/01621459.2015.1123158. URL https://www.tandfonline.com/doi/full/10.1080/01621459.2015.1123158. [p144]

P. J. Diggle, P. Moraga, B. Rowlingson, and B. M. Taylor. Spatial and Spatio-Temporal Log-Gaussian Cox Processes: Extending the Geostatistical Paradigm. *Statistical Science*, 28(4):542–563, nov 2013. ISSN 0883-4237. doi: 10.1214/13-STS441. URL http://projecteuclid.org/euclid.ss/1386078878. [p145, 146]

E. Dovers, J. Stoklosa, and D. I. Warton. Fitting log-gaussian Cox processes using generalized additive model software. *The American Statistician*, 78:418–425, 10 2024. ISSN 0003-1305. doi: 10.1080/00031305.2024.2316725. [p153]

A. O. Finley, A. Datta, B. D. Cook, D. C. Morton, H. E. Andersen, and S. Banerjee. Efficient algorithms for Bayesian nearest neighbor Gaussian processes. *Journal of Computational and Graphical Statistics*, 28:401–414, 4 2019. ISSN 1061-8600. doi: 10.1080/10618600.2018.1537924. [p151]

J. Guinness. Permutation and grouping methods for sharpening Gaussian process approximations. *Technometrics*, 60:415–429, 10 2018. ISSN 0040-1706. doi: 10.1080/00401706.2018.1437476. [p151]

J. S. Hodges and B. J. Reich. Adding Spatially-Correlated Errors Can Mess Up the Fixed Effect You Love. *The American Statistician*, 64(4):325–334, nov 2010. ISSN 0003-1305. doi: 10.1198/tast.2010.10052. URL http://www.tandfonline.com/doi/abs/10.1198/tast.2010.10052. [p160]

O. Johnson, P. Diggle, and E. Giorgi. A spatially discrete approximation to log-gaussian Cox processes for modelling aggregated disease count data. *Statistics in Medicine*, 38:4871–4887, 10 2019. ISSN 0277-6715. doi: 10.1002/sim.8339. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/sim.8339. [p153]

C. M. Jones-Todd and A. B. M. van Helsdingen. stelfi: An R package for fitting hawkes and log-gaussian Cox point process models. *Ecology and Evolution*, 14, 2 2024. ISSN 2045-7758. doi: 10.1002/ece3.11005. [p153]

J. Lessler, A. S. Azman, H. S. McKay, and S. M. Moore. What is a Hotspot Anyway? *The American Journal of Tropical Medicine and Hygiene*, 96(6):1270–1273, jun 2017. ISSN 0002-9637. doi: 10.4269/ajtmh.16-0427. URL http://www.ajtmh.org/content/journals/10.4269/ajtmh.16-0427. [p159]

Y. Li, P. Brown, D. C. Gesink, and H. Rue. Log Gaussian Cox processes and spatially aggregated disease incidence data. *Statistical Methods in Medical Research*, 21:479–507, 10 2012a. ISSN 0962-2802. doi: 10.1177/0962280212446326. [p147, 153]

Y. Li, P. Brown, H. Rue, M. al Maini, and P. Fortin. Spatial modelling of lupus incidence over 40 years with changes in census areas. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 61:99–115, 1 2012b. ISSN 0035-9254. doi: 10.1111/j.1467-9876.2011.01004.x. [p147]

F. Lindgren and H. Rue. Bayesian spatial modelling with R - INLA. *Journal of Statistical Software*, 63, 2015. ISSN 1548-7660. doi: 10.18637/jss.v063.i19. [p152]

C. E. McCulloch. Maximum likelihood algorithms for generalized linear mixed models. *Journal of the American Statistical Association*, 92:162, 3 1997. ISSN 01621459. doi: 10.2307/2291460. [p147]

J. Møller, A. R. Syversveen, and R. P. Waagepetersen. Log Gaussian Cox processes. *Scandinavian Journal of Statistics*, 25:451–482, 9 1998. ISSN 0303-6898. doi: 10.1111/1467-9469.00115. [p149]

F. Palmí-Perales, V. Gómez-Rubio, and M. A. Martinez-Beneito. Bayesian multivariate spatial models for lattice data with INLA. *Journal of Statistical Software*, 98, 2021. ISSN 1548-7660. doi: 10.18637/jss.v098.i02. [p161]

R. Ratnayake, F. Finger, A. S. Azman, D. Lantagne, S. Funk, W. J. Edmunds, and F. Checchi. Highly targeted spatiotemporal interventions against cholera epidemics, 2000–19: a scoping review. *The Lancet Infectious Diseases*, oct 2020a. ISSN 14733099. doi: 10.1016/S1473-3099(20)30479-5. URL https://linkinghub.elsevier.com/retrieve/pii/S1473309920304795. [p144]

R. Ratnayake, F. Finger, W. J. Edmunds, and F. Checchi. Early detection of cholera epidemics to support control in fragile states: estimation of delays and potential epidemic sizes. *BMC Medicine*, 18(1):397, dec 2020b. ISSN 1741-7015. doi: 10.1186/s12916-020-01865-7. URL https://bmcmedicine.biomedcentral.com/articles/10.1186/s12916-020-01865-7. [p144]

B. J. Reich, J. S. Hodges, and V. Zadnik. Effects of Residual Smoothing on the Posterior of the Fixed Effects in Disease-Mapping Models. *Biometrics*, 62(4):1197–1206, dec 2006. ISSN 0006341X. doi: 10.1111/j.1541-0420.2006.00617.x. URL http://doi.wiley.com/10.1111/j.1541-0420.2006.00617.x. [p160]

S. Riley, K. Ainslie, O. Eales, C. Walters, H. Wang, C. Atchinson, C. Fronterre, P. J. Diggle, D. Ashby, C. Donnelly, G. Cooke, W. Barclay, H. Ward, A. Darzi, and P. Elliott. REACT-1 round 6 updated report: high prevalence of SARS-CoV-2 swab positivity with reduced rate of growth in England at the start of November 2020. 2020. URL https://spiral.imperial.ac.uk/handle/10044/1/83912. [p144]

G. Riutort-Mayol, P.-C. Bürkner, M. R. Andersen, A. Solin, and A. Vehtari. Practical hilbert space approximate Bayesian Gaussian processes for probabilistic programming. *Statistics and Computing*, 33:17, 2 2023. ISSN 0960-3174. doi: 10.1007/s11222-022-10167-2. [p151, 152]

H. Rue and L. Held. *Gaussian markov random fields: Theory and applications*. 2005. ISBN 9780203492024. doi: 10.1198/tech.2006.s352. [p152]

D. Simpson, H. Rue, A. Riebler, T. G. Martins, and S. H. Sørbye. Penalising model component complexity: A principled, practical approach to constructing priors. *Statistical Science*, 32, 2 2017. ISSN 0883-4237. doi: 10.1214/16-STS576. [p161]

A. Solin and S. Särkkä. Hilbert space methods for reduced-rank Gaussian process regression. *Statistics and Computing*, 30:419–446, 3 2020. ISSN 0960-3174. doi: 10.1007/s11222-019-09886-w. URL http://link.springer.com/10.1007/s11222-019-09886-w. [p151]

B. M. Taylor and P. J. Diggle. INLA or MCMC? a tutorial and comparative evaluation for spatial prediction in log-gaussian Cox processes. *Journal of Statistical Computation and Simulation*, 84:2266–2284, 10 2014. ISSN 0094-9655. doi: 10.1080/00949655.2013.788653. URL http://www.tandfonline.com/doi/abs/10.1080/00949655.2013.788653. [p152]

B. M. Taylor, T. M. Davies, B. S. Rowlingson, and P. J. Diggle. Lgcp: Inference with spatial and spatio-temporal log-gaussian cox processes in R. *Journal of Statistical Software*, 52:1–40, 2013. ISSN 15487660. [p146, 149, 152]

B. M. Taylor, T. M. Davies, B. S. Rowlingson, and P. J. Diggle. Bayesian inference and data augmentation schemes for spatial, spatiotemporal and multivariate log-gaussian Cox processes in R. *Journal of Statistical Software*, 63, 2015. ISSN 1548-7660. doi: 10.18637/jss.v063.i07. URL http://www.jstatsoft.org/v63/i07/. [p146, 152]

M. Teng, F. Nathoo, and T. D. Johnson. Bayesian computation for log-gaussian Cox processes: a comparative analysis of methods. *Journal of Statistical Computation and Simulation*, 87:2227–2252, 7 2017. ISSN 0094-9655. doi: 10.1080/00949655.2017.1326117. URL https://www.tandfonline.com/doi/full/10.1080/00949655.2017.1326117. [p149, 152]

J. van Niekerk, H. Bakka, H. Rue, and O. Schenk. New frontiers in Bayesian modeling using the INLA package in R. *Journal of Statistical Software*, 100, 2021. ISSN 1548-7660. doi: 10.18637/jss.v100.i02. [p161]

A. V. Vecchia. Estimation and model identification for continuous spatial processes. *Journal of the Royal Statistical Society: Series B (Methodological)*, 50:297–312, 1 1988. ISSN 00359246. doi: 10.1111/j.2517-6161.1988.tb01729.x. [p150]

S. I. Watson, P. J. Diggle, M. G. Chipeta, and R. J. Lilford. Spatiotemporal analysis of the first wave of COVID-19 hospitalisations in Birmingham, UK. *BMJ Open*, 11(10): e050574, oct 2021. ISSN 2044-6055. doi: 10.1136/bmjopen-2021-050574. URL https://bmjopen.bmj.com/lookup/doi/10.1136/bmjopen-2021-050574. [p144]

S. I. Watson, Y. Wang, and E. Giorgi. A fast monte carlo newton-raphson algorithm to estimate generalized linear mixed models with dense covariance. 1 2026. [p147, 148, 164]

1 A minimal, reproducible example

In this section we provide code for a simple, reproducible example using simulated data.

```
b1 <- sf::st_sf(sf::st_sfc(sf::st_polygon(list(cbind(c(0,3,3,0,0),c(0,0,3,3,0))))))
g1 <- grid$new(b1,0.5)
dp <- data.frame(y=runif(10,0,3),x=runif(10,0,3),date=paste0("2021-01-",11:20))
dp <- create_points(dp,pos_vars = c('y','x'),t_var='date')
cov1 <- grid$new(b1,0.8)
cov1$grid_data$cov <- runif(nrow(cov1$grid_data))
g1$add_covariates(cov1$grid_data,
                  zcols="cov")
```

```
g1$points_to_grid(dp, laglength=5)
g1$lgcp_ml(popdens="cov")
fit <- g1$model_fit()
summary(fit)
g1$extract_preds("rr")
g1$hotspots(threshold = 1.2, stat = "rr")

# plot some of the outputs
g1$plot("rr")
g1$plot("hotspot_prob")
```

We can compare our simulated data with the predicted incidence and relative risk.